

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING (CSA1209 – Computer Architecture)

ANNEXURE A – Worked Solutions (CO2)

NAME: M. Dineshumar (192525298)

Question 1 — Signed 8-bit Addition & Subtraction (2's Complement)

Numbers: A = +45, B = -23 (8-bit, 2's complement representation).

Step 1: Binary conversion

- +45 in 8-bit binary = 0 0101101
- 23 in unsigned binary = 00010111
- -23 in 2's complement = invert bits of 00010111 ($\rightarrow$ 11101000) and add 1 $\rightarrow$ 11101001

Step 2: Addition — $A + B = 45 + (-23)$

Align both operands as 8-bit words and add, tracking the carry out of every bit position:

Bit position	7 (MSB)	6	5	4	3	2	1	0 (LSB)
+45	0	0	1	0	1	1	0	1
-23	1	1	1	0	1	0	0	1
Carry generated	1	1	1	1	0	0	1	0
Sum	0	0	0	1	0	1	1	0

Result bits = 00010110₂ = +22 (decimal). A carry of 1 is generated out of the MSB (bit 7); this carry is simply discarded in signed addition (it is not an error by itself).

Check: $45 + (-23) = 22$ ✓

Step 3: Subtraction — $A - B = 45 - (-23)$

Subtraction is performed as addition of the 2's complement of the subtrahend: $A - B = A + (-B)$. Since $B = -23$, $-B = +23 = 00010111$.

Bit position	7 (MSB)	6	5	4	3	2	1	0 (LSB)
+45	0	0	1	0	1	1	0	1
+23	0	0	0	1	0	1	1	1
Carry generated	0	1	1	1	1	1	1	1
Sum	0	1	0	0	0	1	0	0

Result bits = 01000100₂ = +68 (decimal). Check: $45 - (-23) = 45 + 23 = 68$ ✓

Step 4: Overflow detection

A processor detects signed overflow by comparing the carry into the sign bit (C_{in} at bit 7) with the carry out of the sign bit (C_{out} at bit 7):

$$\text{Overflow} = \text{Cin}(\text{MSB}) \text{ XOR } \text{Cout}(\text{MSB})$$

Operation	Carry into bit 7 (Cin)	Carry out of bit 7 (Cout)	Overflow = $\text{Cin} \oplus \text{Cout}$	Interpretation
Addition (45 + (-23))	1	1	0 → No overflow	Result 00010110 = +22, correct and in range [-128,127]
Subtraction (45 - (-23))	0	0	0 → No overflow	Result 01000100 = +68, correct and in range [-128,127]

An alternative, equivalent rule the ALU hardware also uses: overflow occurs only when both operands have the same sign but the result has a different sign. In both operations above this does not happen, so no overflow flag is set and both results (+22 and +68) are correctly represented in 8-bit signed form.

Question 2 — 4-bit Carry Look-Ahead Adder (CLA)

Consider two 4-bit binary inputs A = 1011 (11) and B = 0110 (6), with Cin = 0, added using a 4-bit CLA.

Step 1: Generate (G) and Propagate (P) signals

For each bit position i : $G_i = A_i \cdot B_i$ (a carry is generated at this stage regardless of Cin), and $P_i = A_i \oplus B_i$ (an incoming carry is propagated through this stage).

i	A_i	B_i	$G_i = A_i \cdot B_i$	$P_i = A_i \oplus B_i$
0 (LSB)	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3 (MSB)	1	0	0	1

Step 2: Carries computed in advance (look-ahead equations)

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$

$$C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

$$C_4 (\text{Cout}) = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

Substituting $C_0 = 0$:

- $C_1 = 0 + (1)(0) = 0$
- $C_2 = 1 + (0)(0) + (0)(1)(0) = 1$
- $C_3 = 0 + (1)(1) + (1)(0)(0) + (1)(0)(1)(0) = 1$
- $C_4 = 0 + (1)(0) + (1)(1)(1) + (1)(1)(0)(0) + (1)(1)(0)(1)(0) = 1$

Every carry (C_1, C_2, C_3, C_4) is generated directly from A, B and C_0 through two levels of gates — none of them waits for the carry chain of a ripple adder.

Step 3: Sum bits and final result

$$S_i = P_i \oplus C_i$$

i	P_i	C_i	$S_i = P_i \oplus C_i$
0	1	0	1

i	P _i	C _i	S _i = P _i ⊕ C _i
1	0	0	0
2	1	1	0
3	1	1	0

Sum = S3S2S1S0 = 0001, Cout = C4 = 1 → combined result = 1 0001₂ = 17 decimal. Check: 11 + 6 = 17 ✓

Step 4: Delay comparison — CLA vs Ripple Carry Adder (RCA)

Aspect	Ripple Carry Adder	4-bit Carry Look-Ahead Adder
Carry path	Each stage must wait for Cin from the previous stage	All carries computed directly from A, B, C0 in parallel
Delay (n-bit)	≈ n × (2-gate delay per full adder) → grows linearly with n	≈ constant (2-3 gate levels) regardless of n, plus O(1) extra for the look-ahead logic
For n = 4	≈ 8 gate delays (worst case, carry ripples through all 4 stages)	≈ 3-4 gate delays (G/P generation, then carry logic, then sum XOR)
Hardware cost	Low – simple, repeated full-adder cells	Higher – extra AND/OR gates for the look-ahead network

Justification: In an RCA, stage i cannot compute its sum until C_i arrives from stage i-1, so worst-case delay grows linearly with word length (O(n)). A CLA instead pre-computes every G_i and P_i in one gate delay, and every carry C_i is derived directly from C0 and the G/P signals in a second gate delay — independent of n. This is why modern processors, which need to add wide words (32/64 bit) at GHz clock rates, use look-ahead (or hybrid block-CLA) adders: the reduced, near-constant critical path lets the ALU meet a much shorter clock period than a pure ripple design would allow, at the cost of extra gates.

Question 3 — Booth's Algorithm: (-6) × (+5)

Represent both numbers in 4-bit 2's complement: Multiplicand M = -6 = 1010, Multiplier Q = +5 = 0101. Also compute -M = 0110. Registers: Accumulator A (4 bits, initially 0000), Q (multiplier), Q₋₁ (extra bit, initially 0).

Booth's rule

- Examine the pair (Q0, Q₋₁) at each step.
- 10 → this marks the start of a run of 1's → A = A - M
- 01 → this marks the end of a run of 1's → A = A + M
- 00 or 11 → inside a run of 0's or 1's → no arithmetic operation
- After the add/subtract (or no-op), arithmetic-shift-right the combined register (A, Q, Q₋₁) by 1 bit.

Step-by-step execution (4 cycles, since the multiplier is 4 bits)

Cycle	Q0 Q ₋₁	Operation	A (after op)	A Q Q ₋₁ (after arithmetic shift right)
Initial	1 0	—	0000	A=0000 Q=0101 Q ₋₁ =0
1	1 0	A = A - M = 0000 - 1010	0110	A=0011 Q=0010 Q ₋₁ =1
2	0 1	A = A + M = 0011 + 1010	1101	A=1110 Q=1001 Q ₋₁ =0

Cycle	Q0 Q-1	Operation	A (after op)	A Q Q-1 (after arithmetic shift right)
3	1 0	$A = A - M = 1110 - 1010$	0100	A=0010 Q=0100 Q-1=1
4	0 1	$A = A + M = 0010 + 1010$	1100	A=1110 Q=0010 Q-1=0

Final result

After 4 cycles: Accumulator A = 1110, Q = 0010. The 8-bit product = A concatenated with Q = 1110 0010₂.

Decoding as an 8-bit 2's complement number: invert (0001 1101) and add 1 → 0001 1110 = 30, so the value is -30.

Check: $(-6) \times (+5) = -30$ ✓

Why Booth's algorithm is efficient for signed numbers

- It handles positive and negative multipliers uniformly in 2's complement, with no separate sign-correction step at the end (unlike naive add-and-shift multiplication of signed numbers).
- A run of consecutive 1's in the multiplier (e.g. 0111110) is replaced by just one subtraction at the start of the run and one addition at the end, instead of one addition per 1-bit — this can markedly cut the number of add/subtract operations for multipliers with long runs of 1's.
- Every cycle still performs an arithmetic shift (cheap, single gate-delay per bit) regardless of the bit pattern, so the algorithm's cost is dominated by the (at most one) add/subtract per cycle rather than by every individual bit of the multiplier.

Question 4 — Division: Restoring vs Non-Restoring ($13 \div 3$)

Dividend = 13 = 1101₂, Divisor = 3 = 0011₂, using n = 4-bit operands and a 5-bit remainder register A (initially 0000 0) to accommodate the sign during trial subtraction. M = 00011, -M (2's complement) = 11101.

(a) Restoring Division

Rule for each of the n = 4 cycles: (1) shift the (A, Q) pair left by 1 bit; (2) A = A - M; (3) if A is negative, set Q0 = 0 and restore A by adding M back (A = A + M); otherwise set Q0 = 1 and keep the subtracted A.

Cycle	Shift left A,Q	A - M	A negative?	Action	A after cycle	Q after cycle
1	A=00001, Q=1010	00001-00011=11110	Yes	Q0=0, restore A=A+M	00001	1010
2	A=00011, Q=0100	00011-00011=00000	No	Q0=1, keep A	00000	0101
3	A=00000, Q=1010	00000-00011=11101	Yes	Q0=0, restore A=A+M	00000	1010
4	A=00001, Q=0100	00001-00011=11110	Yes	Q0=0, restore A=A+M	00001	0100

Final: Remainder = A = 00001 = 1, Quotient = Q = 0100 = 4. Check: $13 = 3 \times 4 + 1$ ✓

(b) Non-Restoring Division

Rule: shift (A, Q) left by 1 bit first. Then, if the previous A was non-negative, A = A - M; if the previous A was negative, A = A + M. Set Q0 = 1 if the new A is non-negative, else Q0 = 0 (no explicit restore during the loop). If the final A is negative, a single corrective addition (A = A + M) is performed at the very end.

Cycle	Shift A,Q	Op based on previous A sign	A after op	New A sign	Q0 set	Q after cycle
-------	-----------	-----------------------------	------------	------------	--------	---------------

Cycle	Shift A,Q	Op based on previous A sign	A after op	New A sign	Q0 set	Q after cycle
1	A=00001,Q=1010	prev A $\geq$ 0 $\rightarrow$ A-M	00001-00011=11110	Negative	0	1010
2	A=11101,Q=0100	prev A<0 $\rightarrow$ A+M	11101+00011=00000	Non-neg	1	0101
3	A=00000,Q=1010	prev A $\geq$ 0 $\rightarrow$ A-M	00000-00011=11101	Negative	0	1010
4	A=11011,Q=0100	prev A<0 $\rightarrow$ A+M	11011+00011=11110	Negative	0	0100

Final A (1110_o) is negative $\rightarrow$ one corrective restore: A = A + M = 11110 + 00011 = 00001. Final: Remainder = A = 00001 = 1, Quotient = Q = 0100 = 4. Check: 13 = 3 $\times$ 4 + 1 $\checkmark$ — matching the restoring-division result, as expected.

Comparison: Restoring vs Non-Restoring

Criterion	Restoring Division	Non-Restoring Division
Operations per cycle	Up to 2 (a subtraction, and – whenever the result is negative – an extra restoring addition)	Exactly 1 (either an addition or a subtraction, decided by the previous sign) – no restore inside the loop
Total add/sub ops (this example)	7 (cycles 1,3,4 needed subtract+restore = 2 each; cycle 2 needed 1)	5 (1 op per cycle $\times$ 4, plus 1 final corrective addition)
Control complexity	Simpler condition, but variable/extra operations increase average latency	Slightly more complex control (must remember previous sign), but uniform, predictable 1-op-per-cycle timing
Hardware/speed	Slower on average due to the extra restoring step	Faster and more regular – better suited to pipelined/hardware implementation

Non-restoring division is generally preferred in modern processors because it performs exactly one addition or subtraction per cycle (never two), giving a fixed, predictable cycle time that is easier to pipeline in hardware, whereas restoring division's extra conditional 'restore' step adds variable overhead and extra energy/time on every cycle where the trial subtraction fails.

Question 5 — IEEE 754 Representation and Addition (–13.25)

Step 1: Convert –13.25 to binary and normalize

- 13 (integer part) = 1101₂
- 0.25 (fractional part) = 01₂ (0.25 = 1 $\times$ 2⁻²)
- So 13.25 = 1101.01₂
- Normalized (scientific) form: 1101.01 = 1.10101 $\times$ 2³
- Sign bit = 1 (number is negative)

Step 2: Single precision (32-bit: 1 sign, 8 exponent, 23 mantissa)

- Exponent field uses bias 127: stored exponent = actual exponent + bias = 3 + 127 = 130 = 10000010₂
- Mantissa (fraction after the implicit leading 1): 10101, padded with zeros to 23 bits $\rightarrow$ 10101000000000000000000

Sign (1 bit)	Exponent (8 bits) — bias 127	Mantissa (23 bits)
--------------	------------------------------	--------------------

is called catastrophic cancellation, and it is the main source of precision loss in floating-point addition of like-magnitude, opposite-sign numbers.

- Alignment shift: shifting the smaller operand's mantissa right during exponent equalization can also shift bits below the mantissa's least-significant position (into 'guard/round/sticky' bits in real hardware); if not tracked, this too becomes a source of rounding error.